

Object Detection

Object Detection

1. Environment Setup
 - 1.1 File Transfer
 - 1.2 Install the Driver Library
2. Model Introduction
3. Object Prediction: Image
 - Effect Preview
4. Object Prediction: Video
 - Effect Preview
5. Object Prediction: Real-Time Detection
 - CSI Camera
 - Effect Preview

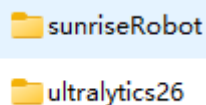
Use Python to demonstrate the effects of **Ultralytics**: Object Detection on images, videos, and real-time detection.

Before running this case, if your image is the one we shipped that can already drive the IMX415, you can directly perform the following operations. If it is not an IMX415 image, you need to refer to the tutorial on driving the IMX415 to make sure it can be driven, and then proceed with the environment setup operations.

1. Environment Setup

1.1 File Transfer

First, transfer the `sunriseRobot` folder from the rdk source code summary to the motherboard. The `ultralytics26` folder, which contains the driver program for the yolo26 case, also needs to be transferred in.



1.2 Install the Driver Library

After the transfer, enter the following command to enter the directory:

```
cd /home/sunrise/sunriseRobot/SunriseRobotLib
```

If your motherboard has a similar driver library, first uninstall the old driver library:

```
sudo pip uninstall SunriseRobotLib -y
```

Then install:

```
sudo pip install -e
```

2. Model Introduction

Object detection is a task that involves identifying the location and category of objects in an image or video stream.

The output of an object detector is a set of bounding boxes that enclose the objects in the image, along with the class label and confidence score for each box. If you need to identify objects of interest in a scene without knowing the exact location or precise shape of the objects, object detection is a good choice.

3. Object Prediction: Image

Use the `yolo26n_ncnn_model1` model to predict the images in the `ultralytics26` folder.

Enter the code folder:

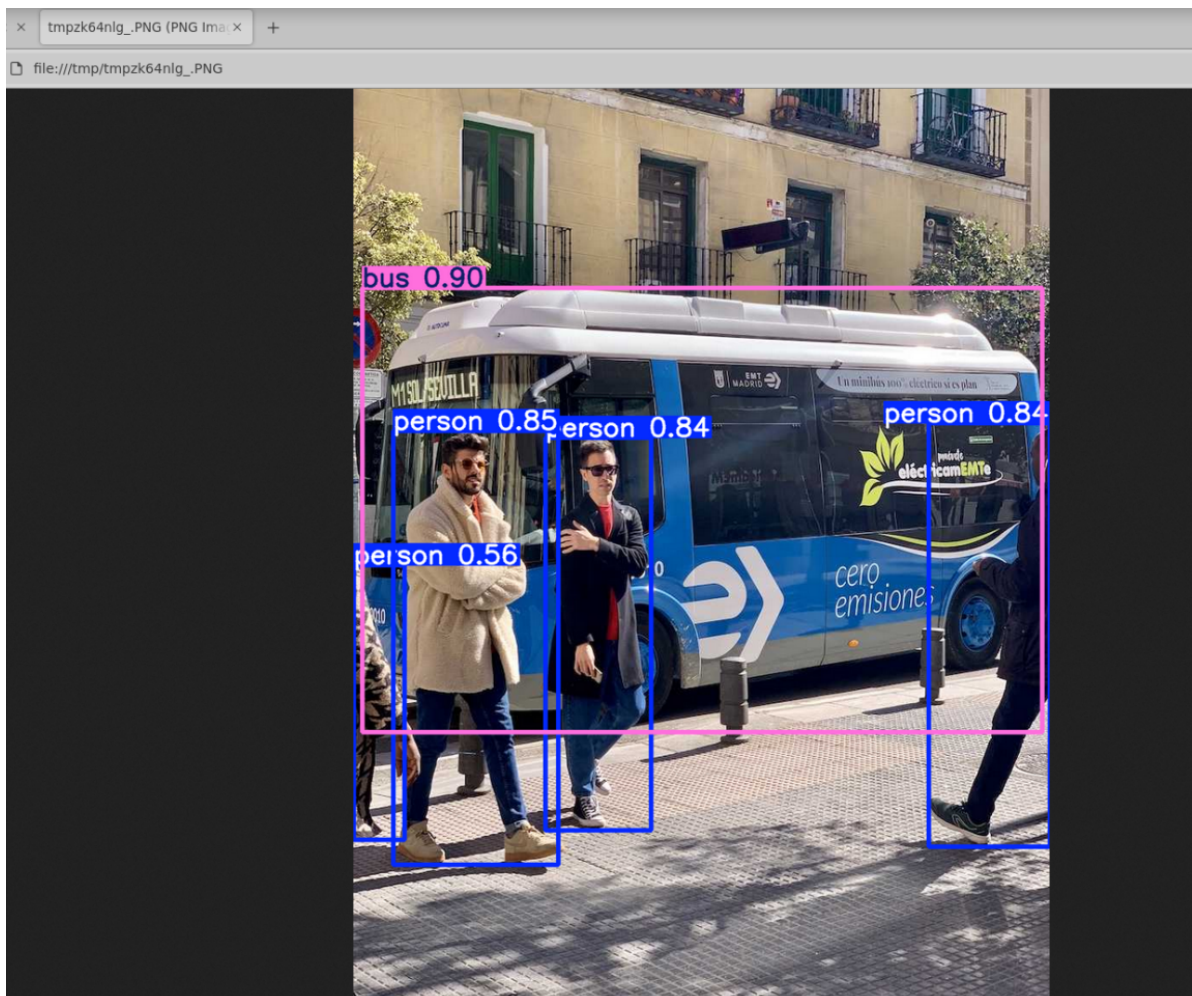
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 01.detection_image.py
```

Effect Preview

The location of the image output by yolo recognition: `/home/sunrise/ultralytics26/output/`



Example code:

```

from ultralytics import YOLO

# Load a model
model = YOLO("/home/sunrise/ultralytics26/yolo26n_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n.pt")

# Run batched inference on a list of images
results = model("/home/sunrise/ultralytics26/assets/bus.jpg") # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/home/sunrise/ultralytics26/output/bus_output.jpg") #
    save to disk

```

4. Object Prediction: Video

Use the `yolo26n_ncnn_model` model to predict the videos in the `ultralytics26` folder (not the videos that come with ultralytics).

Enter the code folder:

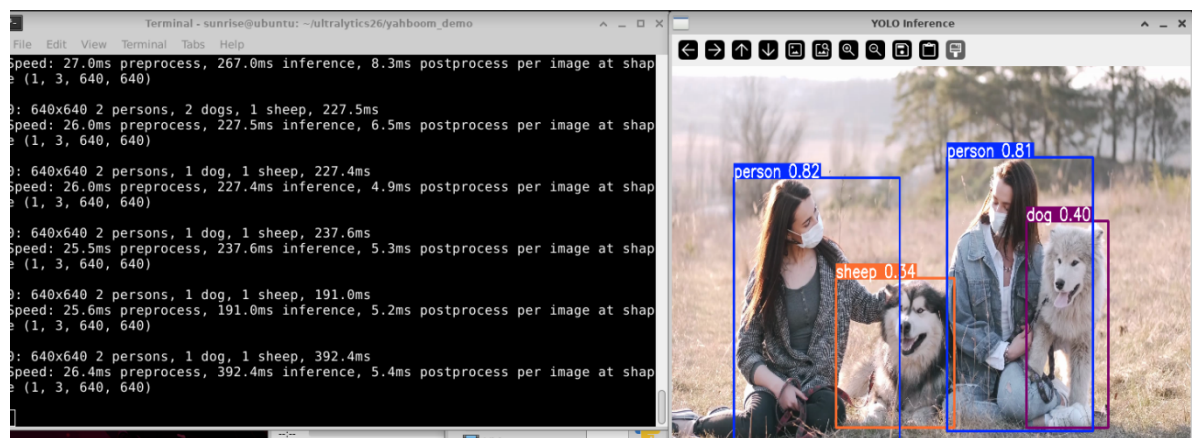
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 01.detection_video.py
```

Effect Preview

The location of the video output by yolo recognition: `/home/sunrise/ultralytics26/output/`



Example code:

```

import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/sunrise/ultralytics26/yolo26n_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n.pt")

# Open the video file
video_path = "/home/sunrise/ultralytics26/videos/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/01.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()

```

```
out.release()  
cv2.destroyAllWindows()
```

5. Object Prediction: Real-Time Detection

CSI Camera

Use the `yolo26n_ncnn_model` model to predict the CSI camera image.

Enter the code folder:

```
cd /home/sunrise/ultralytics26/yahboom_demo
```

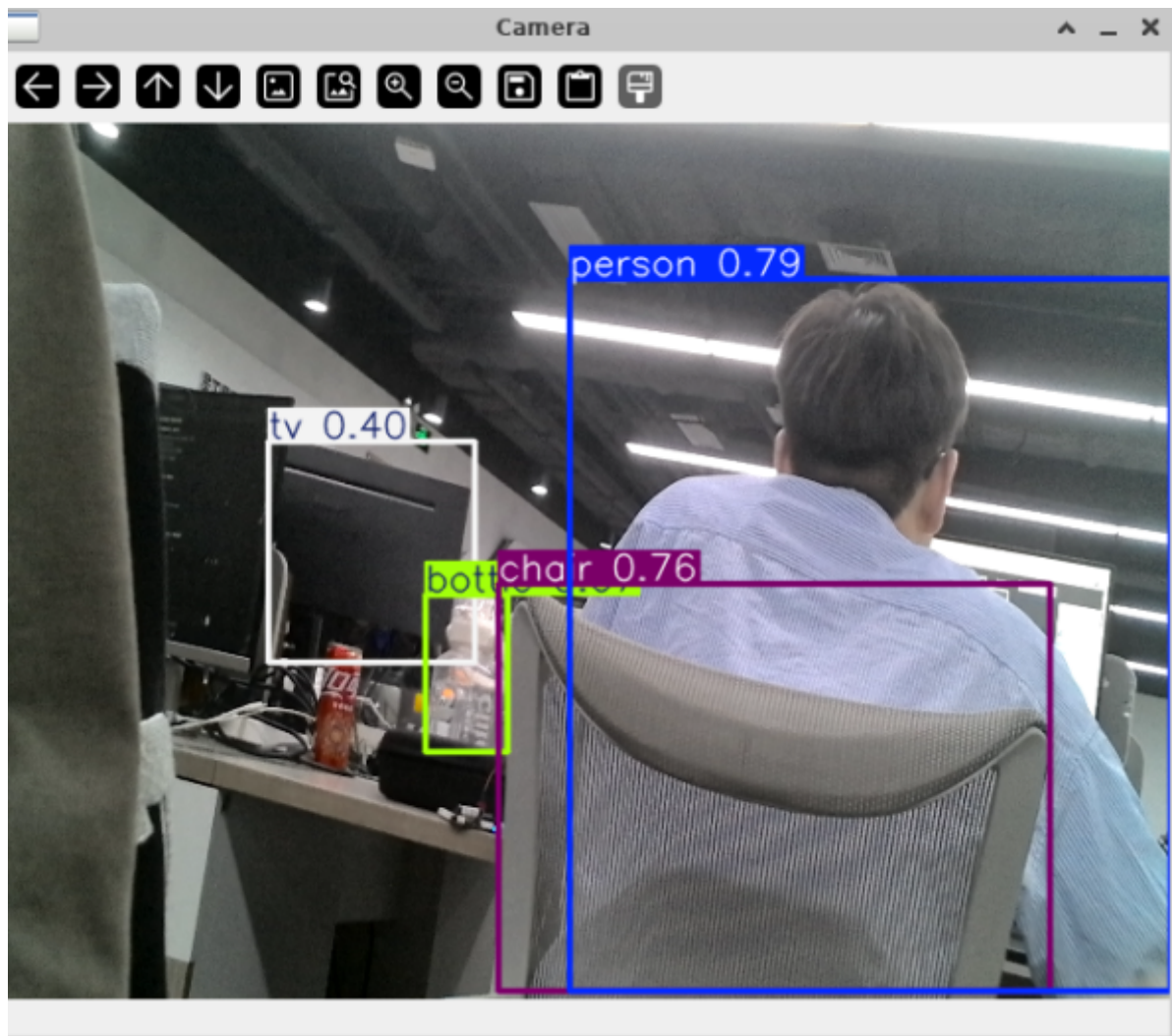
Run the code: click to preview the image

```
python3 01.detection_camera_csi.py
```

```
: 640x640 (no detections), 3723.4ms  
speed: 17.3ms preprocess, 3723.4ms inference (1, 3, 640, 640)
```

Effect Preview

The location of the video output by yolo recognition: `/home/sunrise/ultralytics26/output/`



Example code:

```
import cv2
from ultralytics import YOLO
from SunriseRobotLib import Mipi_Camera

camera = Mipi_Camera(width=3840, height=2160, debug=True)

model = YOLO("/home/sunrise/ultralytics26/yolo26n_ncnn_model", task='detect')

FLIP_CODE = -1

# Frame counter, used for frame skipping to optimize lag
frame_id = 0

while True:
    ret, frame = camera.read()
    if not ret:
        print("Read failed, check the camera")
        break
    frame_resized = cv2.resize(frame, (640, 480))
    frame_flipped = cv2.flip(frame_resized, FLIP_CODE)
    frame_id += 1
    if frame_id % 2 != 0:

        results = model(frame_flipped)
        annotated = results[0].plot()
```

```
    else:
        annotated = frame_flipped

    cv2.imshow("YOLO", annotated)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

camera.release()
cv2.destroyAllWindows()
```